

SystemVerilog: Inheritance, Virtual Function and Polymorphism

1. Base Class – first

```
class first;
    int data=12;
    virtual function void display();
        $display("FIRST:value of data is: %0d",data);
    endfunction
endclass
```

The class first is the base (parent) class. It contains a data member data initialized to 12 and a virtual function display().

2. Derived Class – second

```
class second extends first;
    int temp=34;
    function void add();
        $display("Second:value after process is : %0d",temp+4);
    endfunction

    function void display();
        $display("Second:value of data :%0d",data);
    endfunction
endclass
```

The class second is derived from first using extends. Therefore, second inherits the data member data and the display() function from first.

The second class also defines its own display() function. This is called function overriding.

It also has its own member temp and an additional function add().

3. Class Handles and Object Creation

```
first f;
second s;

f=new();
s=new();
```

f and s are class handles. The new() statements create objects.

Conceptually:

```
f  —————> first object
                  data = 12

s  —————> second object
                  data = 12
                  temp = 34
```

4. Base-Class Handle Pointing to Derived-Class Object

```
f=s;
```

This assignment is allowed because a base-class handle can point to a derived-class object.

After f=s:

```
f  ———┐
        │
        ▼
    second object
      data = 12
      temp = 34
```

The handle f is declared as type first, but it now refers to a second-class object.

The original first object that f was pointing to is no longer referenced by f.

5. Calling the Virtual Function

```
f.display();
```

This is the most important part.

Although f is declared as a first-class handle, it is currently pointing to a second-class object.

Because display() was declared as virtual in the base class, SystemVerilog performs dynamic dispatch. It checks the actual object at runtime and calls the overridden function belonging to that object.

Therefore:

```
Handle type      → first
```

Actual object → second
Function called → second.display()

Output:

Second:value of data :12

6. Why is virtual Important?

With virtual:

```
virtual function void display();
```

```
f.display();
```

calls second.display(), because f points to a second object.

If virtual is removed:

```
function void display();
```

the function call is resolved according to the handle type. Since f is declared as first, f.display() calls first.display().

Output without virtual:

FIRST:value of data is: 12

7. Simple Comparison

Without virtual:

Handle type → first

Function called → first.display()

With virtual:

Handle type → first

Actual object → second

Function called → second.display()

8. Important Interview Point

A virtual function allows a base-class handle pointing to a derived-class object to invoke the derived-class overridden function at runtime.

This is called runtime polymorphism or dynamic polymorphism.

9. Important Note About add()

`f.add();` is not legal.

Although `f` is actually pointing to a second object, its handle type is `first`. The first class does not contain an `add()` function.

Therefore, functions that exist only in the derived class cannot be directly accessed through a base-class handle.

To access `add()`, use:

```
s.add();
```

because `s` is a second-class handle.